parse_fastq.py 사용법 및 코드 설명서

대상 파일: `lab11/src/parse_fastq.py`

1. 프로그램의 목적

FASTQ 파일을 읽어서 각 read를 (이름, 서열, 품질) 묶음으로 저장한 뒤, 다음 네 가지를 화면에 출력한다.

- read 개수
- 첫 번째 read의 이름
- 첫 번째 read의 서열
- 첫 번째 read의 길이

2. 실행 방법

`lab11` 폴더에서 다음과 같이 실행한다.

```
python3 src/parse_fastq.py data/SRR1553607_1.fastq
```

형식:

```
python3 src/parse_fastq.py <FASTQ 파일>
```

인자로 FASTQ 파일 경로 하나를 준다.

3. 입력 파일

FASTQ 형식 파일이다. FASTQ는 read 하나를 **4줄**로 표현한다.

줄	내용	예 (data/SRR1553607_1.fastq의 첫 read)
1	@ + read 이름	@SRR1553607.1 1 length=101
2	염기 서열	GTTAGCGTTGTTGATCGCG...
3	+ (구분 줄)	+SRR1553607.1 1 length=101
4	품질 점수 문자열	BB@FFFFFHHHHHJJJJ...

이 코드는 파일이 정확히 4줄 단위로 되어 있다고 가정하고 읽는다.

4. 출력 결과

`data/SRR1553607_1.fastq`로 실행한 실제 결과:

```
read 개수: 203445
첫 read 이름: SRR1553607.1 1 length=101
첫 read 서열:
GTTAGCGTTGTTGATCGCGACGCAACAACTGGTAAAGAATCTGGAAGAAGGATATCAGTTCAAACGCTCAAGCGAGATGATGGATATTTTTGAACGACTCA
첫 read 길이: 101
```

- 이름에는 맨 앞의 `@`가 빠져 있다 (`[1:]` 때문, 아래 설명 참고).
- 품질 문자열도 저장은 되지만 화면에는 출력하지 않는다.

5. 전체 코드

```
import sys

def read_fastq(filename):
    records = []
    with open(filename) as f:
        lines = f.readlines()
```

```
    for i in range(0, len(lines), 4):
        name = lines[i].rstrip()[1:]
        seq = lines[i + 1].rstrip()
        qual = lines[i + 3].rstrip()
        records.append((name, seq, qual))

    return records

reads = read_fastq(sys.argv[1])
print("read 개수:", len(reads))
print("첫 read 이름:", reads[0][0])
print("첫 read 서열:", reads[0][1])
print("첫 read 길이:", len(reads[0][1]))
```

6. `read_fastq()` 함수의 동작 원리

1. 결과를 담을 빈 리스트 `records`를 만든다.
2. `open(filename)`으로 파일을 열고 `readlines()`로 모든 줄을 한꺼번에 읽어 리스트 `lines`에 저장한다. `with` 블록이 끝나면 파일은 자동으로 닫힌다.
3. `for i in range(0, len(lines), 4)`로 `i`를 0, 4, 8, ...처럼 4씩 늘리며 read 하나(4줄)씩 처리한다.
   - `lines[i]` → 1번째 줄(이름 줄)
   - `lines[i + 1]` → 2번째 줄(서열)
   - `lines[i + 2]` → 3번째 줄(+ 줄): **코드에서 사용하지 않고 건너뛴다.**
   - `lines[i + 3]` → 4번째 줄(품질)
4. 이름 줄은 `rstrip()`으로 줄바꿈을 지운 뒤 `[1:]`로 맨 앞의 `@`를 뺀다. 서열과 품질 줄은 `rstrip()`만 한다.
5. `(name, seq, qual)` 튜플을 `records`에 `append()`로 추가한다.
6. 반복이 끝나면 `records`를 돌려준다.

따라서 반환값은 다음과 같은 리스트이다.

```
[
  ("SRR1553607.1 1 length=101", "GTTAGCG...", "BB@FFFF..."),
  ("SRR1553607.2 2 length=101", "GGTGTAA...", "?@;BDDD..."),
  ...
]
```

메인 부분에서 `reads[0]`은 첫 번째 튜플, `reads[0][0]`은 그 이름, `reads[0][1]`은 그 서열이다.

7. 주요 문법 설명

`readlines()`

파일의 모든 줄을 읽어 **문자열 리스트**로 돌려준다. 각 원소 끝에는 줄바꿈 문자 `\n`이 붙어 있다.

`lines[0]` → `"@SRR1553607.1 1 length=101\n"`

파일 전체를 한 번에 메모리에 올리기 때문에, 이후 `lines[i + 1]`처럼 번호로 원하는 줄에 바로 접근할 수 있다.

`range(0, len(lines), 4)`

0부터 `len(lines)` 바로 전까지 **4씩 증가**하는 숫자를 만든다. 줄 수가 8이면 `0, 4`가 나온다. FASTQ의 read 하나가 4줄이므로, `i`는 항상 각 read의 첫 줄(`@` 줄) 위치를 가리킨다.

`rstrip()`

문자열 **오른쪽 끝**의 공백 문자(줄바꿈 `\n` 포함)를 지운다.

`"GTTAGCG\n".rstrip()` → `"GTTAGCG"`

이것을 하지 않으면 서열 길이에 `\n`이 1글자로 더 세어진다.

`[1:]`

문자열의 **두 번째 글자(인덱스 1)부터 끝까지** 잘라 낸다(슬라이싱). 이름 줄의 맨 앞 `@`를 없애는 데 쓴다.

`"@SRR1553607.1 1 length=101"[1:]` → `"SRR1553607.1 1 length=101"`

코드에서는 `lines[i].rstrip()[1:]`처럼 먼저 `rstrip()`으로 줄바꿈을 지운 뒤 `[1:]`을 적용한다.

append()

리스트 **맨 끝에 원소 하나를 추가**한다. 여기서는 `(name, seq, qual)` 튜플 하나를 `records`에 넣는다. 괄호가 두 겹인 이유는 바깥 괄호가 `append()` 호출, 안쪽 괄호가 튜플이기 때문이다.

sys.argv[1]

`sys.argv`는 명령줄에 입력한 단어들의 리스트이다.

```
python3 src/parse_fastq.py data/SRR1553607_1.fastq
```

- `sys.argv[0]` → `src/parse_fastq.py` (스크립트 이름)
- `sys.argv[1]` → `data/SRR1553607_1.fastq` (첫 번째 인자)

따라서 `read_fastq(sys.argv[1])`은 사용자가 입력한 FASTQ 파일을 읽는다. 이 때문에 맨 위에 `import sys`가 있다.

8. 주의할 점 (현재 코드의 동작 그대로)

코드에는 오류 검사 부분이 없으므로 다음 경우에는 오류가 난다.

- 인자 없이 실행하면 `sys.argv[1]`이 없어서 `IndexError`가 난다.
- 파일의 줄 수가 4의 배수가 아니면 마지막 read에서 `lines[i + 1]` 또는 `lines[i + 3]`이 없어 `IndexError`가 난다.
- 빈 파일이면 `reads`가 빈 리스트가 되어 `reads[0]`에서 `IndexError`가 난다.
- 서열이 여러 줄로 나뉜 FASTQ는 4줄 규칙이 깨지므로 제대로 읽지 못한다.
- `readlines()`로 파일 전체를 한 번에 읽으므로 파일이 매우 크면 메모리를 많이 쓴다.